CS 271P · Artificial Intelligence · Fall 2026

PA 3D: Monte-Carlo Tree Search

Submission Instructions

You will build a Monte-Carlo Tree Search agent and submit it to Gradescope, where it plays a tournament against our agents. Your grade is how often it beats them. This handout is the complete specification.

At a Glance

- **Submit on Gradescope:** exactly one file, `agent.py` (this exact name) — the file itself, not a folder, archive, or notebook.
- **Edit** any of the four MCTS phase methods (`select`, `expand`, `simulate`, `backpropagate`) in the provided `Agent` class, using the **Python 3 standard library only**. Do not edit `game.py` or `mcts_framework.py`.
- **Graded by how often your agent BEATS our hidden agents**, up to **105%**. Note: our agents get **25% more thinking time per move** than yours.

THE ASSIGNMENT

You are given a complete but deliberately **naive** MCTS agent in `agent.py`: random selection, all-children expansion, uniformly random play-outs, and standard back-propagation. It plays legally, but weakly. **Your job is to make it as strong as you can** by rewriting any of its four phase methods (and adding any helpers or `__init__` state you like).

The game your agents play is **"King of the Hill" Los Alamos chess** — the 6×6 game from the PA 3 notebooks (simplified piece moves, no bishops), with one added win condition. A side wins in **either** of two ways:

- by **capturing the enemy king**, *or*
- by moving **its own king onto one of the four central squares** — $(2,2)$, $(2,3)$, $(3,2)$, $(3,3)$.

The game ends the instant either happens; a long game with neither is a draw. It is a two-player, constant-sum game. Every agent runs inside the *same* search framework; the only thing that differs between agents is those four phase methods. The engine permits moves into attack: it uses king capture rather than check or checkmate. There is no castling, double pawn step, or en passant; pawns promote to queen, rook, or knight.

THE FOUR PHASES YOU CAN CHANGE

- `select(node, rng)` — which already-expanded child to descend into. *Naive:* a random child.
- `expand(node, game, rng)` — grow the tree at the frontier and return a child to simulate from. *Naive:* add all children, return one at random.
- `simulate(game, state, rng)` — a quick play-out estimating who wins from `state`; return `'w'` / `'b'` / `None`. *Naive:* a uniformly random game.
- `backpropagate(node, winner, game)` — push the play-out result up the path. *Standard:* credit each node from the perspective of the player who moved into it (a draw counts $\frac{1}{2}$).

A ready random play-out helper, `random_playout(game, state, rng)`, is available. The move actually played each turn is the **most-visited** child of the root.

WHERE TO LOOK

- **The game** (`game.py`): read its public interface — `actions` / `result` / `is_terminal` / `winner` / `to_move` — to see what your agent can observe about a position.
- **Your MCTS lecture notes** and the shipped survey, *A Survey of Monte-Carlo Tree Search Methods* (Browne et al., 2012), which covers many enhancements you can study and adapt.

RESTRICTIONS

> **Important:** Submissions that break these rules may fail to load and score **0**.
>
> - The file must be named `agent.py` and define a class `Agent` with the four methods above (same names and signatures). It must instantiate with `Agent()` (no required arguments).
> - **Python 3 standard library only** — no `numpy`, no third-party packages. (`math`, `random`, `collections`, etc. are fine.)
> - Do **not** edit or submit `game.py` or `mcts_framework.py`; we grade with our own copies. Your agent must return a **legal** move within its time budget.
> - Do **not** read files, access the network, or spam `print`. No global state between moves.

TEST BEFORE YOU SUBMIT

The package includes `test_local.py`. From the assignment folder, run:

```
python3 test_local.py           # plays your agent vs the naive baseline (~1 minute)
python3 test_local.py 0.05      # smaller time budget = faster, for quick iteration
python3 test_local.py --final # more public-baseline games; prints a fresh seed
python3 test_local.py --final --seed 12345 # repeat the opening inputs
python3 test_local.py --list  # numbered openings and named public positions
python3 test_local.py --board "near hill, White" # inspect one chosen move
```

It plays your agent against the **naive baseline** you started from — the same opponent as the 45% tier — under the same 25%-more-time handicap the real references get. The plain command keeps the quick six-opening test. The `-final` option uses 36 mixed-depth openings (72 games), drawn with a fresh local seed; `-seed N` repeats their inputs and random streams. Timed searches may still choose differently. It uses only the public baseline, so this is practice rather than a prediction of the full grade.

Use `-list` to see those openings plus nine illustrative public positions; it works even if your agent cannot load. Use `-board 5` or a quoted name to inspect one move: the board, legal moves, any immediate wins, your move, and its resulting board and outcome. These two options are alternatives. **PASS means legal output**, not a best-move or grade guarantee; missing an available immediate win is an observation, not a phase-correctness failure.

The optional **PA 3 Agent Lab** notebook shows your `agent.py`'s chosen move and a short match replay. Its visual packages are optional; the command-line test and your submitted agent use only the standard library. Edit and submit `agent.py` only.

> **Important:** A great local score is **necessary but not sufficient** for full marks: it does not include our three stronger hidden agents, which earn the remaining 60%.

HOW YOUR WORK IS GRADED

Your agent plays a tournament against **four of our hidden agents**. Your score is by how often you **beat** each one, where **beat%** counts each drawn game as one half:

- **45%** — in proportion to how often you beat our **naive random** agent.
- **+20%** — in proportion to how often you beat a **stronger** agent.
- **+25%** — in proportion to how often you beat a **much stronger** agent.
- **+15%** — in proportion to how often you beat our **strongest** agent (**up to 105%** total).

Important: Our agents get **25% more thinking time per move** than yours. So simply copying an *equal* agent tends to **lose** — to win you must be smarter, or more efficient, per sample. Beating the naive agent is the foundation, but it is only 45% — the remaining **60%** comes from our three stronger agents, so a working agent that stops there leaves a lot on the table.

Two grading runs. The regular grader gives feedback on a fresh random sample on each run. After every submission and extension is in, the **FINAL** grader regrades everyone on more positions from the same opening distributions, with a fixed seed shared across students. It uses the same game, opponents, thinking-time calibration, handicap, and weights. **The final run's score is your assignment grade**: out of 50 points, with a 52.5-point (105%) cap. Each result identifies the run, seed, and sample sizes. A fixed seed repeats opening inputs and random streams; searches limited by CPU time can complete different iteration counts, so repeated scores can still vary.

Grading environment. Your code runs on Gradescope in a **Linux (Ubuntu)** container under **Python 3** with **no third-party packages** — the same way `test_local.py` runs it here. Test with the provided game and framework; grading can use different positions. You may resubmit as often as you like before the deadline.

SUBMITTING ON GRADESCOPE

1. Open the `PA 3D: Monte-Carlo Tree Search` assignment on **Gradescope**.
2. Upload your `agent.py` (the file only).
3. Wait for the tournament to finish, then read your per-opponent results and score.
4. Resubmit as often as you like before the deadline.

These instructions are subject to change. Any updates will be announced in class and posted on Canvas.